Advanced Methods in Deep Learning

Lecture 2: Non-linearities, Loss Functions, Backpropagation & Gradient Descent

Alexander Quispe Rojas
Universidad del Pacífico – MECA
aw.quispero@up.edu.pe aquisper@caltech.edu

April 17, 2026

Today's Roadmap

Recap and Activation Functions Deep Dive

Loss Functions in Detail

The Chain Rule

Backpropagation Algorithm

Gradient Descent Variants

PyTorch Implementation

Summary and Next Steps

Recap & Activation Functions

Recap from Lecture 1

What we learned:

- A neural network computes: $\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$.
- The **forward pass** maps input $\mathbf{x}$ to prediction $\hat{y}$.
- We need a **loss function** $\mathcal{L}$ to measure how bad our prediction is.
- We use **gradient descent** to update parameters: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \mathcal{L}$.

Today's question:

How do we compute $\nabla_{\mathbf{w}} \mathcal{L}$ efficiently for a deep network?

Answer: The **backpropagation** algorithm — a systematic application of the chain rule.

Activation Functions: Derivatives Matter

To do backpropagation, we need the **derivative** of each activation function.

Name	$\sigma(z)$	$\sigma'(z)$	**Range**
Sigmoid	$\frac{1}{1+e^{-z}}$	$\sigma(z)(1-\sigma(z))$	$(0,1)$
Tanh	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$1-\tanh^2(z)$	$(-1,1)$
ReLU	$\max(0,z)$	1 if $z>0$; 0 if $z<0$	$[0,\infty)$
Leaky ReLU	$\max(\alpha z,z)$	1 if $z>0$; α if $z<0$	$(-\infty,\infty)$

Deriving the Sigmoid Derivative

Let $\sigma(z) = \frac{1}{1+e^{-z}} = (1+e^{-z})^{-1}$. Apply the chain rule:

Step 1: $\sigma'(z) = -(1+e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2}$

Step 2: Rewrite in terms of $\sigma(z)$:

$$\sigma'(z) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z) \cdot \frac{(1+e^{-z})-1}{1+e^{-z}} = \sigma(z)(1-\sigma(z))$$

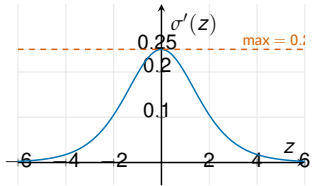
Result

$$\boxed{\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)}$$

Note: Max value $\sigma'(0) = 0.25$. This causes the vanishing gradient problem.

The Vanishing Gradient Problem

Sigmoid derivative peaks at 0.25:



Problem for deep networks:
With L layers of sigmoid, the gradient at layer 1 involves:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} \propto \prod_{l=1}^{L} \sigma'(z^{(l)})$$

Since $\sigma'(z) \leq 0.25$:

$$\prod_{l=1}^{L} 0.25 = 0.25^{L}$$

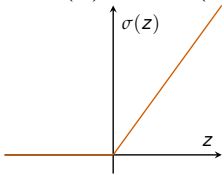
For $L = 10$: $0.25^{10} \approx 10^{-6}$

Gradients vanish exponentially with depth!

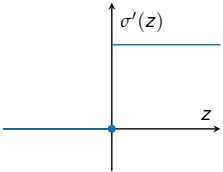
Solution: Use **ReLU**, where $\sigma'(z) = 1$ for $z > 0$. No vanishing gradient!

Why ReLU Dominates

ReLU: $\sigma(z) = \max(0, z)$



ReLU derivative:



- Derivative $= 1$ for $z > 0$: no vanishing gradient.
- Computationally cheap.
- Sparse activation (many neurons output 0).

Problem: "Dead ReLU" — if $z < 0$ always, the neuron never updates.

Fix: Leaky ReLU ($\alpha = 0.01$), ELU, GELU.

Softmax: From Binary to Multi-class Classification

For K classes, we need a probability distribution over classes.

Softmax Function

Given logits $z_1, z_2, \ldots, z_K$ (raw outputs of the network), the **softmax** function computes:

$$p(y = k|\mathbf{x}) = \text{softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \qquad \text{for } k = 1, \ldots, K$$

Properties:

- $\sum_{k=1}^{K} p(y = k|\mathbf{x}) = 1$ (valid probability distribution).
- $p(y = k|\mathbf{x}) > 0$ for all k (always positive).
- Larger $z_k \Rightarrow$ higher probability for class k.
- When $K = 2$: softmax reduces to the sigmoid function.

Softmax: Numerical Example

Network output (logits) for $K = 3$ classes:

$$\mathbf{z} = \begin{pmatrix} z_1 \\ z_2 \\ z_3 \end{pmatrix} = \begin{pmatrix} 2.0 \\ 1.0 \\ 0.1 \end{pmatrix}$$

Step 1: Compute exponentials:

$$e^{z_1} = e^{2.0} = 7.389, \quad e^{z_2} = e^{1.0} = 2.718, \quad e^{z_3} = e^{0.1} = 1.105$$

Step 2: Sum of exponentials:

$$\sum_{j=1}^{3} e^{z_j} = 7.389 + 2.718 + 1.105 = 11.212$$

Step 3: Normalize:

$$p_1 = \frac{7.389}{11.212} = 0.659, \quad p_2 = \frac{2.718}{11.212} = 0.242, \quad p_3 = \frac{1.105}{11.212} = 0.099$$

Prediction: class 1 with probability 65.9%. Check: $0.659 + 0.242 + 0.099 = 1.0$ ✓

Loss Functions in Detail

Loss Functions: Overview

The loss function $\mathcal{L}$ measures how far our prediction $\hat{y}$ is from the truth y.

Task	**Loss**	**Formula**	**Output activation**
Regression	MSE	$\frac{1}{N}\sum(y_i - \hat{y}_i)^2$	None (linear)
Binary classif.	BCE	$-\frac{1}{N}\sum[y \log \hat{y} + (1-y)\log(1-\hat{y})]$	Sigmoid
Multi-class	CE	$-\frac{1}{N}\sum\sum y_{ik} \log \hat{y}_{ik}$	Softmax

Key principle: the choice of loss function depends on the task. The loss determines what the network optimizes for.

MSE Loss: Gradient Derivation

Loss for one sample: $\ell = (y - \hat{y})^2$ where $\hat{y} = f(\mathbf{x}; \theta)$.

Gradient with respect to the prediction:

$$\frac{\partial \ell}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}}(y - \hat{y})^2 = 2(y - \hat{y}) \cdot (-1) = -2(y - \hat{y})$$

Average over N samples:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = -\frac{2}{N}(y_i - \hat{y}_i)$$

Interpretation:

- The gradient is proportional to the **error** $(y_i - \hat{y}_i)$.
- Large errors produce large gradients $\Rightarrow$ faster learning.
- This is the **starting point** of backpropagation: we first compute $\frac{\partial \mathcal{L}}{\partial \hat{y}}$, then propagate backward.

Binary Cross-Entropy: Derivation from MLE

Assume $y \in \{0, 1\}$ and model predicts $\hat{y} = p(y = 1|\mathbf{x})$. Likelihood of one sample: $p(y|\mathbf{x}) = \hat{y}^y (1 - \hat{y})^{1-y}$.

Log-likelihood $\rightarrow$ **Negative log-likelihood** (= BCE):

$$\ell = -\big[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\big]$$

Gradient w.r.t. $\hat{y}$**:**

$$\frac{\partial \ell}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

Gradient w.r.t. logit z (where $\hat{y} = \sigma(z)$, $\sigma' = \sigma(1 - \sigma)$):

$$\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial \hat{y}} \cdot \sigma'(z) = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \cdot \hat{y}(1 - \hat{y}) = \hat{y} - y$$

The Beautiful Cancellation: BCE + Sigmoid

Key Result

$$\frac{\partial \ell}{\partial z} = \sigma(z) - y = \hat{y} - y$$

The gradient of BCE w.r.t. the logit z is simply $\hat{y} - y$.

Why is this beautiful?

- No $\sigma'(z)$ term in the gradient!
- Even when sigmoid saturates, gradient is large if prediction is wrong.
- **Not a coincidence:** BCE was designed to pair with sigmoid.
- Compare: MSE + sigmoid $\Rightarrow \frac{\partial \ell}{\partial z} = -2(y - \hat{y})\sigma'(z)$ – vanishes!

Lesson: Always pair the right loss with the right activation.

Categorical Cross-Entropy Loss

For K-class classification with one-hot encoding $\mathbf{y} = (y_1, \ldots, y_K)$:

Categorical Cross-Entropy

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log \hat{y}_k = -\log \hat{y}_c$$

where c is the true class (since only $y_c = 1$, all other $y_k = 0$).

Example: True class $= 2$ (i.e., $\mathbf{y} = (0, 1, 0)$), predictions $\hat{\mathbf{y}} = (0.1, 0.7, 0.2)$:

$$\mathcal{L} = -(0 \cdot \log 0.1 + 1 \cdot \log 0.7 + 0 \cdot \log 0.2) = -\log(0.7) = 0.357$$

If predictions were $\hat{\mathbf{y}} = (0.1, 0.95, 0.05)$:

$$\mathcal{L} = -\log(0.95) = 0.051 \quad \text{(much lower} - \text{better prediction)}$$

Interpretation: The loss only depends on the **predicted probability for the correct class**.

Softmax + Cross-Entropy: Gradient (Step 1)

Let $\hat{y}_k = \text{softmax}(z_k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$ and $\mathcal{L} = -\sum_k y_k \log \hat{y}_k$.

We want: $\frac{\partial \mathcal{L}}{\partial z_i}$ for each logit z_i. First, compute the **Jacobian of softmax**:

Case $i = k$:

$$\frac{\partial \hat{y}_k}{\partial z_k} = \frac{e^{z_k} \sum_j e^{z_j} - e^{z_k} \cdot e^{z_k}}{(\sum_j e^{z_j})^2} = \hat{y}_k - \hat{y}_k^2 = \hat{y}_k(1 - \hat{y}_k)$$

Case $i \neq k$:

$$\frac{\partial \hat{y}_k}{\partial z_i} = \frac{0 - e^{z_k} \cdot e^{z_i}}{(\sum_j e^{z_j})^2} = -\hat{y}_k \hat{y}_i$$

Compact form: $\frac{\partial \hat{y}_k}{\partial z_i} = \hat{y}_k(\delta_{ik} - \hat{y}_i)$ where δ_{ik} is the Kronecker delta.

Softmax + Cross-Entropy: Gradient (Step 2)

Step 2: Apply chain rule with $\mathcal{L} = -\sum_k y_k \log \hat{y}_k$:

$$\frac{\partial \mathcal{L}}{\partial z_i} = \sum_{k=1}^{K} \frac{\partial \mathcal{L}}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_i} = -\sum_{k=1}^{K} \frac{y_k}{\hat{y}_k} \cdot \hat{y}_k(\delta_{ik} - \hat{y}_i)$$

Simplify:

$$= -\sum_k y_k(\delta_{ik} - \hat{y}_i) = -y_i + \hat{y}_i \underbrace{\sum_k y_k}_{=1} = \hat{y}_i - y_i$$

Key Result

$$\boxed{\frac{\partial \mathcal{L}}{\partial z_i} = \hat{y}_i - y_i \qquad \text{(prediction minus truth)}}$$

Same result as BCE + sigmoid! This elegant cancellation is by design.

The Chain Rule

The Chain Rule: Single Variable

If $y = f(u)$ and $u = g(x)$, then:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Example: Let $y = (3x + 2)^2$.

- Let $u = 3x + 2$, so $y = u^2$.
- $\frac{dy}{du} = 2u = 2(3x + 2)$.
- $\frac{du}{dx} = 3$.
- $\frac{dy}{dx} = 2(3x + 2) \cdot 3 = 6(3x + 2)$.

Longer chains: If $y = f(g(h(x)))$:

$$\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dx}$$

This is exactly what happens in a neural network: each layer is a function composition.

The Chain Rule: Multiple Variables

If $\mathcal{L} = f(y_1, y_2)$ where $y_1 = g_1(x)$ and $y_2 = g_2(x)$:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y_1}\frac{\partial y_1}{\partial x} + \frac{\partial \mathcal{L}}{\partial y_2}\frac{\partial y_2}{\partial x}$$

In general, if $\mathcal{L}$ depends on x through $y_1, y_2, \ldots, y_m$:

$$\frac{\partial \mathcal{L}}{\partial x} = \sum_{j=1}^{m} \frac{\partial \mathcal{L}}{\partial y_j}\frac{\partial y_j}{\partial x}$$

Example (relevant to neural networks):

A weight $w_{13}^{(1)}$ in layer 1 affects multiple hidden neurons, which all affect the loss:

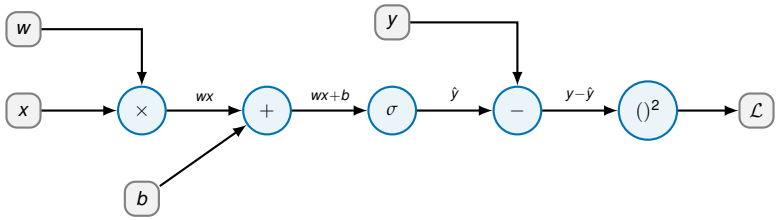
$$\frac{\partial \mathcal{L}}{\partial w_{13}^{(1)}} = \sum_j \frac{\partial \mathcal{L}}{\partial h_j^{(2)}} \cdot \frac{\partial h_j^{(2)}}{\partial w_{13}^{(1)}}$$

We need to sum contributions from all paths through the network.

Computational Graphs

A **computational graph** represents a function as a sequence of operations.

Example: $\mathcal{L} = (y - \sigma(wx + b))^2$



- **Forward pass:** compute output left to right.
- **Backward pass:** compute gradients right to left using chain rule.

Backward Pass Through the Graph

Starting from $\mathcal{L} = (y - \hat{y})^2$, let $e = y - \hat{y}$. Work right-to-left:

$$\frac{\partial \mathcal{L}}{\partial e} = 2e = 2(y - \hat{y}) \qquad \text{(derivative of square)}$$

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial \mathcal{L}}{\partial e} \cdot \frac{\partial e}{\partial \hat{y}} = 2(y - \hat{y}) \cdot (-1) = -2(y - \hat{y}) \qquad \text{(chain rule)}$$

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \sigma'(z) = -2(y - \hat{y}) \cdot \sigma'(z) \qquad \text{(through activation)}$$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot x = -2(y - \hat{y})\, \sigma'(z)\, x \qquad \text{(through multiplication)}$$

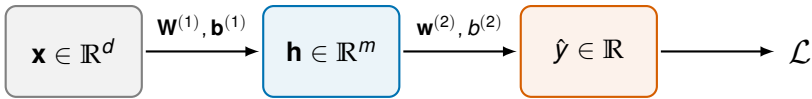
Key pattern: each node passes its gradient backward, multiplied by the local derivative.

Backpropagation

The engine of deep learning

Backpropagation: Setup

Consider a **2-layer network** (1 hidden layer):



Forward pass equations:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \qquad \mathbf{z}^{(1)} \in \mathbb{R}^m \quad \text{(pre-activation)}$$

$$\mathbf{h} = \sigma(\mathbf{z}^{(1)}) \qquad \mathbf{h} \in \mathbb{R}^m \quad \text{(activation)}$$

$$z^{(2)} = (\mathbf{w}^{(2)})^\top \mathbf{h} + b^{(2)} \qquad z^{(2)} \in \mathbb{R} \quad \text{(output logit)}$$

$$\hat{y} = z^{(2)} \qquad \text{(for regression, no final activation)}$$

$$\mathcal{L} = (y - \hat{y})^2 \qquad \text{(MSE loss)}$$

Goal: Compute $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}}, \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}}, \frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(2)}}, \frac{\partial \mathcal{L}}{\partial b^{(2)}}$.

Backpropagation Step 1: Output Layer Gradients

Start from the loss and work backward.

Step 1a: Gradient of loss w.r.t. output:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}}(y - \hat{y})^2 = -2(y - \hat{y})$$

Since $\hat{y} = z^{(2)}$: $\frac{\partial \mathcal{L}}{\partial z^{(2)}} = -2(y - \hat{y})$. Define $\delta^{(2)} \equiv \frac{\partial \mathcal{L}}{\partial z^{(2)}} = -2(y - \hat{y})$.

Step 1b: Gradient w.r.t. output layer parameters:

$$z^{(2)} = \sum_{j=1}^{m} w_j^{(2)} h_j + b^{(2)}$$

$$\frac{\partial \mathcal{L}}{\partial w_j^{(2)}} = \delta^{(2)} \cdot \frac{\partial z^{(2)}}{\partial w_j^{(2)}} = \delta^{(2)} \cdot h_j \qquad \Rightarrow \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(2)}} = \delta^{(2)} \cdot \mathbf{h}$$

$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = \delta^{(2)}$$

Backpropagation Step 2: Hidden Layer Gradients

Step 2a: Propagate error back to hidden layer. Using chain rule:

$$\frac{\partial \mathcal{L}}{\partial h_j} = \delta^{(2)} \cdot \frac{\partial z^{(2)}}{\partial h_j} = \delta^{(2)} \cdot w_j^{(2)}$$

Step 2b: Pass through the activation function:

$$\frac{\partial \mathcal{L}}{\partial z_j^{(1)}} = \frac{\partial \mathcal{L}}{\partial h_j} \cdot \sigma'(z_j^{(1)}) = \delta^{(2)} \cdot w_j^{(2)} \cdot \sigma'(z_j^{(1)}) \equiv \delta_j^{(1)}$$

Step 2c: Gradients w.r.t. hidden layer parameters (since $z_j^{(1)} = \sum_k W_{jk}^{(1)} x_k + b_j^{(1)}$):

$$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(1)}} = \delta_j^{(1)} \cdot x_k \quad \Rightarrow \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top \qquad \frac{\partial \mathcal{L}}{\partial b_j^{(1)}} = \delta_j^{(1)}$$

The General Backpropagation Formula

Define the **error signal** at layer l: $\boldsymbol{\delta}^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$

Backpropagation Equations

Output layer ($l = L$): $\boldsymbol{\delta}^{(L)} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \odot \sigma'(\mathbf{z}^{(L)})$

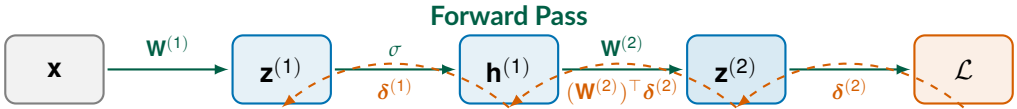
Hidden layers ($l = L-1, \ldots, 1$): $\boldsymbol{\delta}^{(l)} = \left[(\mathbf{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)}\right] \odot \sigma'(\mathbf{z}^{(l)})$

Parameter gradients: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{h}^{(l-1)})^\top \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$

Here $\odot$ denotes element-wise (Hadamard) product and $\mathbf{h}^{(0)} = \mathbf{x}$.

Intuition: The error at layer l is the error from layer $l+1$ projected back through $\mathbf{W}^{(l+1)}$, scaled by how sensitive the activation was (σ').

Backpropagation: Visual Summary



Algorithm:

1. **Forward:** Compute all $\mathbf{z}^{(l)}$, $\mathbf{h}^{(l)}$, and $\mathcal{L}$ (save intermediate values).
2. **Backward:** Compute $\boldsymbol{\delta}^{(L)}, \boldsymbol{\delta}^{(L-1)}, \ldots, \boldsymbol{\delta}^{(1)}$ using the recursion.
3. **Update:** $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha\, \boldsymbol{\delta}^{(l)} (\mathbf{h}^{(l-1)})^\top$ for all l.

Backpropagation: Numerical Example (Part 1)

Network: 1 input $\to$ 1 hidden (sigmoid) $\to$ 1 output (linear). MSE loss.

Parameters: $w^{(1)} = 0.5$, $b^{(1)} = 0.1$, $w^{(2)} = -0.3$, $b^{(2)} = 0.2$
Data: $x = 2.0$, $y = 1.0$. **Learning rate:** $\alpha = 0.1$.

Forward pass:

$$z^{(1)} = w^{(1)}x + b^{(1)} = 0.5 \cdot 2.0 + 0.1 = 1.1$$

$$h = \sigma(z^{(1)}) = \sigma(1.1) = \frac{1}{1 + e^{-1.1}} = 0.7503$$

$$z^{(2)} = w^{(2)}h + b^{(2)} = -0.3 \cdot 0.7503 + 0.2 = -0.0251$$

$$\hat{y} = z^{(2)} = -0.0251$$

$$\mathcal{L} = (y - \hat{y})^2 = (1.0 - (-0.0251))^2 = (1.0251)^2 = 1.0508$$

Backpropagation: Numerical Example (Part 2)

Backward pass (work right-to-left through the network):

Step 1 – Error at output:

$$\delta^{(2)} = -2(y - \hat{y}) = -2(1.0 - (-0.0251)) = -2.0502$$

Step 2 – Output layer gradients:

$$\frac{\partial \mathcal{L}}{\partial w^{(2)}} = \delta^{(2)} \cdot h = (-2.0502)(0.7503) = -1.5383 \qquad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = \delta^{(2)} = -2.0502$$

Step 3 – Error at hidden layer:

$$\delta^{(1)} = \delta^{(2)} \cdot w^{(2)} \cdot \sigma'(z^{(1)}) = (-2.0502)(-0.3)(0.7503)(1 - 0.7503) = 0.1154$$

Step 4 – Hidden layer gradients:

$$\frac{\partial \mathcal{L}}{\partial w^{(1)}} = \delta^{(1)} \cdot x = (0.1154)(2.0) = 0.2307 \qquad \frac{\partial \mathcal{L}}{\partial b^{(1)}} = \delta^{(1)} = 0.1154$$

Backpropagation: Numerical Example (Part 3)

Parameter update with learning rate $\alpha = 0.1$: $\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot \text{grad}$

Parameter	Old	Gradient	New
$w^{(2)}$	-0.3000	-1.5383	-0.1462
$b^{(2)}$	0.2000	-2.0502	0.4050
$w^{(1)}$	0.5000	0.2307	0.4769
$b^{(1)}$	0.1000	0.1154	0.0885

Verify: Forward pass with new params $\rightarrow \hat{y}_{\text{new}} = 0.2969$

$$\mathcal{L}_{\text{new}} = (1.0 - 0.2969)^2 = 0.4946 \quad \text{(down from 1.0508! ✓)}$$

One gradient step reduced the loss by 53%. Repeat this process thousands of times to train the network.

Vector and Matrix Calculus Notation

In practice, we work with **matrices**, not individual weights.

Key identities (for layer l, with $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{h}^{(l-1)})^\top \qquad \in \mathbb{R}^{m^{(l)} \times m^{(l-1)}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \qquad \in \mathbb{R}^{m^{(l)}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(l-1)}} = (\mathbf{W}^{(l)})^\top \boldsymbol{\delta}^{(l)} \qquad \in \mathbb{R}^{m^{(l-1)}}$$

Why outer product? Each $\delta_j^{(l)}$ pairs with each $h_k^{(l-1)}$: $\frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}} = \delta_j^{(l)} \cdot h_k^{(l-1)}$.

The gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$ has the same shape as $\mathbf{W}^{(l)}$.

Computational Cost of Backpropagation

- **Forward pass cost:** $O\left(\sum_{l=1}^{L} m^{(l)} m^{(l-1)}\right)$ (matrix-vector products).
- **Backward pass cost:** same order as forward pass.
- **Memory cost:** must store all $\mathbf{z}^{(l)}, \mathbf{h}^{(l)}$ during forward pass.
- **Total:** backprop is roughly **2x** the cost of the forward pass.

Historical Significance

Before backprop (1986): computing gradients for a network with P parameters required $O(P)$ separate forward passes.
With backprop: all gradients in one backward pass.

Gradient Descent Variants

Batch Gradient Descent

Full batch gradient descent uses **all** N samples to compute the gradient:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{w}} \ell(\mathbf{x}_i, y_i; \mathbf{w})$$

Pros:

- Stable convergence, low noise.
- Exact gradient direction.

Cons:

- Requires loading **entire dataset** into memory.
- Very slow for large N (one update per epoch).
- Can get stuck in local minima (smooth trajectory).

Not practical when N is large (e.g., ImageNet: $N = 1.2$ million).

Stochastic Gradient Descent (SGD)

SGD uses a **single random sample** per update:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \nabla_{\mathbf{w}} \ell(\mathbf{x}_i, y_i; \mathbf{w}) \qquad \text{where } i \text{ is sampled randomly}$$

Pros:

- Very fast updates.
- Can escape shallow local minima (noise helps!).
- Low memory requirement.

Cons:

- Very noisy gradient estimate.
- Oscillates around minimum.
- May never converge exactly.

Key insight: SGD's gradient is an **unbiased estimate** of the true gradient:

$$\mathbb{E}_i \left[\nabla_{\mathbf{w}} \ell(\mathbf{x}_i, y_i; \mathbf{w}) \right] = \frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{w}} \ell(\mathbf{x}_i, y_i; \mathbf{w}) = \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

Mini-Batch SGD: The Best of Both Worlds

Mini-batch SGD uses a random subset $\mathcal{B} \subset \{1, \ldots, N\}$ of size B:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\mathbf{w}} \ell(\mathbf{x}_i, y_i; \mathbf{w})$$

Method	Batch size	Noise	Speed
Batch GD	$B = N$	None	Slow
SGD	$B = 1$	High	Fast
Mini-batch SGD	$B = 32, 64, 128, 256$	Moderate	Fast

Why mini-batches work so well:

- GPUs handle parallel matmul — 32 samples $\approx$ as fast as 1.
- Noise reduced by factor $\frac{1}{\sqrt{B}}$ vs. pure SGD.
- Typical: $B = 32, 64, 128, 256$ (powers of 2 for GPU efficiency).

Epochs, Iterations, and Batch Size

Terminology

- **Epoch:** one complete pass through the entire dataset.
- **Iteration:** one parameter update (= one mini-batch).
- **Iterations per epoch:** $\lceil N/B \rceil$.

Example: $N = 10{,}000$ training samples, batch size $B = 100$:

- Iterations per epoch: $10{,}000/100 = 100$.
- If we train for 50 epochs: $50 \times 100 = 5{,}000$ total parameter updates.

Training recipe:

1. Shuffle the dataset.
2. Split into mini-batches of size B.
3. For each mini-batch: forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update.
4. Repeat for multiple epochs.

SGD with Momentum

Pure SGD oscillates due to noisy gradients. **Momentum** smooths the trajectory:

Momentum Update

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t) \qquad \mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{v}_t$$

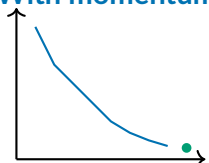
where $\beta \in [0, 1)$ is the momentum coefficient (typically $\beta = 0.9$).

Intuition: a ball rolling down a hill. Momentum accumulates past gradients, so it "rolls through" small bumps.

Without momentum

Oscillates, slow convergence.

With momentum



Smoother, faster convergence.

PyTorch Implementation

Backpropagation in PyTorch: Under the Hood

PyTorch does backpropagation automatically via `autograd`.

```python
import torch
import torch.nn as nn

# 2-layer network: 3 inputs -> 4 hidden (ReLU) -> 1 output
model = nn.Sequential(
    nn.Linear(3, 4),      # W1: 4x3, b1: 4 -> 16 params
    nn.ReLU(),
    nn.Linear(4, 1))      # W2: 1x4, b2: 1 -> 5 params (Total: 21)

# Forward pass + loss
x = torch.randn(1, 3); y_true = torch.tensor([[1.0]])
loss = nn.MSELoss()(model(x), y_true)

# Backward pass -- ALL gradients computed automatically!
loss.backward()

for name, param in model.named_parameters():
    print(f"{name}: grad shape = {param.grad.shape}")
```

Full Training Example: Economic Data

```python
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

# Simulated economic data: 3 features -> outcome
X = torch.randn(1000, 3)
y = (2*X[:,0] - X[:,1] + 0.5*X[:,2] + torch.randn(1000)*0.1).unsqueeze(1)

# Mini-batch DataLoader
loader = DataLoader(TensorDataset(X, y), batch_size=32, shuffle=True)

# Model, loss, optimizer
model = nn.Sequential(nn.Linear(3, 16), nn.ReLU(),
                      nn.Linear(16, 8), nn.ReLU(), nn.Linear(8, 1))
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

# Training loop
for epoch in range(50):
    for X_batch, y_batch in loader:
        loss = criterion(model(X_batch), y_batch)
        optimizer.zero_grad(); loss.backward(); optimizer.step()
```

Visualizing the Training Loss

```
import matplotlib.pyplot as plt

losses = []
for epoch in range(100):
    epoch_loss = 0
    for X_batch, y_batch in loader:
        loss = criterion(model(X_batch), y_batch)
        optimizer.zero_grad(); loss.backward(); optimizer.step()
        epoch_loss += loss.item()
    losses.append(epoch_loss / len(loader))

plt.plot(losses); plt.xlabel('Epoch'); plt.ylabel('MSE Loss')
plt.title('Training Loss Curve'); plt.grid(True); plt.show()
```

What to look for: loss should decrease; oscillations $\Rightarrow$ LR too large; slow decrease $\Rightarrow$ LR too small.

Summary & Next Steps

Lecture 2 Summary

1. **Activation function derivatives** are critical for backprop. Sigmoid: $\sigma' = \sigma(1-\sigma)$. ReLU: $\sigma' = \mathbb{1}[z > 0]$.
2. **Vanishing gradients:** sigmoid derivative ≤ 0.25 causes exponential decay in deep nets. ReLU solves this.
3. **Softmax** extends binary classification to K classes: $\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$.
4. **Cross-entropy + softmax/sigmoid** gives gradient $\frac{\partial \mathcal{L}}{\partial z_i} = \hat{y}_i - y_i$.
5. **Backpropagation** = chain rule applied layer by layer:

$$\boldsymbol{\delta}^{(l)} = [(\mathbf{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)}] \odot \sigma'(\mathbf{z}^{(l)}) \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{h}^{(l-1)})^\top$$

6. **Mini-batch SGD** with momentum is the standard optimizer. Typical $B = 32$–256.

Next Lecture: SGD, Regularization & Generalization

Lecture 3 (Monday, April 20):

- Advanced optimizers: Adam, AdaGrad, RMSProp.
- Regularization: L1, L2 (weight decay), dropout.
- Batch normalization.
- Universal Approximation Theorem; why depth matters.
- Practical tips: LR schedules, early stopping.

Recommended reading: Goodfellow et al., *Deep Learning*, Ch. 6.5 + Ch. 8. Zhang et al., *D2L*, Ch. 5 (`https://d2l.ai`).

Homework 1 will be released after Lecture 3 (due Sunday, April 27).

Questions?

References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapters 6, 8.

- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into Deep Learning*. Cambridge University Press.

- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.

- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS*.

- Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. *ICML*.

- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR*.